降りタイマー (oriTimer) 要件定義書

要件定義書とは？

要件定義書は設計書よりもさらに **上流** のドキュメントです。
「何を作るか」を決める前に「なぜ作るのか」「誰のために作るのか」「どんな課題を解決するのか」を定義します。

ドキュメントの位置づけ:

```
要件定義書      →  基本設計書      →  詳細設計書      →  実装
（なぜ作るか）     （何を作るか）     （どう作るか）     （コードを書く）
```

観点	要件定義書	基本設計書	
主な読者	発注者·経営者·PM	PM·開発リーダー	開発者
記述レベル	ビジネス·業務の言葉	機能とシステムの言葉	技術·実装の言葉
主な内容	目的·課題·要望	機能一覧·画面設計	クラス·関数·DB定義
例	「駅に着いたら知らせてほしい」	「F-01: 路線·駅一覧表示機能」	「NativeGeofenceM

なぜ要件定義書が必要か：

要件定義がないまま設計·実装を始めると、以下のような問題が起きます。

- 「思ってたのと違う」 → 手戻りが発生する
- 「あの機能も欲しい」 → 際限なく機能が増える（スコープクリープ）
- 「そもそも何のために作ったんだっけ」 → プロジェクトの方向性を見失う

要件定義書の各セクションの役割:

セクション	書く内容	なぜ必要か
1. プロジェクト背景	なぜこのプロジェクトを始めるのか	判断に迷ったとき「原点」に戻れるよ

セクション	書く内容	なぜ必要か
2. 目的とゴール	何を達成すれば成功なのか	完了条件を明確にするため
3. ステークホルダー	誰が関係者か	意思決定者と利用者を明確にするため
4. 現状の課題	今何が困っているのか	﹁解決すべき問題﹂を全員で共有する
5. 業務フロー	ユーザーがどう使うか	実際の利用シーンを想像するため
6. 機能要件	何ができればよいか	開発範囲の合意を取るため
7. 非機能要件	どの程度の品質が必要か	﹁動くけど使えない﹂を防ぐため
8. 制約条件	守るべきルール·制限	技術選定や制約の前提を共有するため
9. リスク	何が問題になりそうか	事前に対策を打つため
10. 受入条件	何をもって完成とするか	﹁終わり﹂を定義するため

---

1. プロジェクト背景

> **このセクションの目的：**﹁このプロジェクトはなぜ始まったのか﹂を記述する。背景を書くことで、開発中に判断に迷ったとき﹁そもそもの目的は何だったか﹂に立ち返ることができる。

1.1 背景

電車通勤·通学において、座席に座っていると居眠りしてしまい、降りるべき駅を乗り越すことがある。時刻指定の目覚ましでは対応しにくい理由として、以下がある。

- 電車の遅延·運休により、時刻が一定でない
- 乗り換えが多いルートでは、目的駅の到着時刻を正確に予測しにくい
- 毎回アラームをセットし直す手間がある

スマートフォンの **ジオフェンス機能**（特定の地点に近づいたときにイベントを発火する仕組み）を活用することで、時刻ではなく﹁場所（駅）﹂をトリガーとしたアラームを実現できる

と考えた。

同時に、開発者自身のスキル向上のため、以下の技術を実践的に学習する機会を求めていた。

- Flutter によるモバイルアプリ開発
- ネイティブジオフェンスAPIのFlutterからの利用
- Riverpod / freezed 等の Flutter エコシステム
- バックグラウンド動作を伴うアプリの設計

これらの課題と学習目標を同時に満たすプロジェクトとして、本アプリの開発を開始した。

1.2 プロジェクト名

降りタイマー **(oriTimer)**

---

2. 目的とゴール

> **このセクションの目的**：プロジェクトの成功基準を定義する。「何ができたら完成なのか」が曖昧だと、永遠に終わらないプロジェクトになってしまう。

2.1 ビジネス目的

> **補足**：個人開発であっても「ビジネス目的」を書くのは良い習慣。企業のプロジェクトでは「売上を10%向上する」「問い合わせを50%削減する」のような数値目標が入る。

#	目的
1	東京の電車に乗っているとき、降りるべき駅に近づいたら自動で通知を受け取れるようにする
2	時刻ではなく「場所」をトリガーとすることで、遅延・乗り換えに左右されない乗り越し防止を実
3	Flutter・ジオフェンスAPI・Riverpod の実践的な開発スキルを習得する

2.2 達成指標

> **補足**：達成指標は、できるだけ **客観的に判定できるもの** にする。「使いやすい」は主観的で判定できないが、「3タップ以内で設定完了できる」は客観的に判定できる。

#	指標	達成基準
1	駅一覧表示	東京の主要路線の駅が路線ごとにまとめて表示される
2	設定の到達性	駅選択から監視開始まで3タップ以内で完了できる
3	通知の発火	選択した駅の半径1000m圏内に入ったときに通知が発火される
4	バックグラウンド動作	アプリを閉じた状態でもジオフェンス監視·通知が機能する
5	学習成果	Riverpod / freezed / native_geofence を活用した実装がされている

---

3. ステークホルダー

> **このセクションの目的**：プロジェクトに関わる﹁人﹂を明確にする。
>
> **補足**：ステークホルダーとは﹁利害関係者﹂のこと。企業プロジェクトでは多くの関係者がいるが、個人開発でも﹁誰が使うのか﹂﹁誰が決定権を持つのか﹂を整理しておくと、判断が早くなる。

役割	担当	責任
プロジェクトオーナー	開発者本人	要件の決定、優先順位の
開発者	開発者本人 + AI（Claude）	設計·実装·テスト
エンドユーザー	開発者本人（将来的に知人や通勤者にも共有）	アプリの利用、フィード

---

4. 現状の課題

> **このセクションの目的**：﹁今、何が困っているのか﹂を書く。課題が明確であれば、その課題を解決する機能だけを作ればよいので、ムダな開発を避けられる。

4.1 課題一覧

課題ID	カテゴリ	課題内容
P-01	乗り越し	電車内で居眠りして降りるべき駅を乗り越してしまう
P-02	利便性	時刻指定のアラームは電車の遅延に対応できない
P-03	利便性	乗り換えが多い場合、目的駅の到着時刻を事前に正確に把握しにくい
P-04	利便性	毎日アラームを設定し直す手間がかかる
P-05	学習	Flutter・ジオフェンスAPI・バックグラウンド処理の実践的な学習機会がほしい

4.2 課題と機能の対応

> **補足：** この対応表を作ることで「この機能はどの課題を解決するために作るのか」が明確になる。課題に紐づかない機能は不要な可能性がある。

課題ID	解決する機能
P-01	ジオフェンス監視、降車通知
P-02	場所ベースのトリガー（時刻ではなく位置情報で通知）
P-03	駅選択による自動通知（何時に着くかを意識しなくてよい）
P-04	一度設定すれば継続的に機能するジオフェンス監視
P-05	プロジェクト全体（技術スタックの選定）

---

5. 業務フロー（ユースケース）

> **このセクションの目的：** ユーザーが実際にどのようにアプリを使うかを、具体的なシナリオで描く。
>
> **補足：** 「ユースケース」とは「利用者がシステムを使って何かを達成する一連の流れ」のこと。画面の細かいデザインではなく、ユーザーの「行動」に注目して書く。

UC-01: 降りる駅を設定してアラームを開始する

```
目的:        電車乗車前に、降りる駅のジオフェンス監視を開始する
利用者:      エンドユーザー
事前条件:    アプリがインストールされ、初回起動済みである

基本フロー:
  1. ユーザーがアプリを起動する
  2. システムが路線・駅の一覧を取得・表示する
  3. ユーザーが駅名を検索フォームに入力し、該当する路線へジャンプする（任意）
  4. ユーザーが降りたい駅を選択する
  5. ユーザーが監視開始ボタンをタップする
  6. システムが位置情報パーミッションを確認する
  7. システムがジオフェンスを登録し、監視を開始する
  8. ユーザーが電車に乗る（アプリをバックグラウンドにする）

代替フロー:
  3a. 検索結果に該当駅がない場合
      → 「該当する駅が見つかりませんでした」とダイアログに表示する
  6a. パーミッションが未取得の場合
      → システムが「パーミッション要求」ボタンへの操作を促す
  6b. 「常に許可」が付与されていない場合
      → 設定画面に誘導してパーミッション取得を促す

例外フロー:
  2a. APIサーバーに接続できない場合
      → エラーメッセージを表示する
```

UC-02: 駅に近づいて降車通知を受け取る

```
目的:        選択した駅の近くに到達したことを通知で知る
利用者:      エンドユーザー
事前条件:  UC-01が完了し、ジオフェンス監視が起動中である

基本フロー:
  1. ユーザーが電車に乗り、選択した駅へ向かう（アプリはバックグラウンド）
  2. スマートフォンのOSが駅の半径1000m圏内への入場を検知する
  3. システムが降車通知を発火する（音量最大化（Android）+ ループバイブレーション + プッシュ通知）
  4. バイブレーションはユーザーが停止操作をするまで繰り返し続ける
  5. ユーザーが通知を確認し、電車を降りる準備をする
  6. ユーザーがアプリの停止ボタンを押すと、バイブレーションが止まる

代替フロー:
  2a. OSの省電力設定等によりジオフェンスイベントが遅延する場合
      → 遅延してでもイベント発火を試みる（即時性の保証はできない）

例外フロー:
  3a. 通知パーミッションがない場合
      → 通知は表示されない（バイブレーションのみ）
```

UC-03: 監視を停止する

```
目的:        不要になったジオフェンス監視を解除する
利用者:      エンドユーザー
事前条件:   ジオフェンス監視が起動中である

基本フロー（マルチゴール0件または1件の場合）:
  1. ユーザーがアプリを起動する
  2. ユーザーが「停止（×）」ボタンをタップする
  3. システムが登録済みの全ジオフェンスを削除する
  4. システムがループ中のバイブレーションを停止する
  5. システムが監視停止状態に切り替える
  6. システムが選択駅の情報を画面・SharedPreferencesの両方から削除する
  7. マルチゴールが1件登録されていた場合、そのエントリも削除する

代替フロー（マルチゴールが2件以上の場合）:
  2a. ユーザーが「more_vert（メニュー）」ボタンをタップする
  2b. 登録済みマルチゴールのポップアップが表示される
  2c. 最後に登録したゴールのみ×ボタン付きで表示される（削除可能）
  2d. ユーザーが×の付いた項目をタップすると、そのゴールのジオフェンスとエントリを削除する

事後条件: 以後、停止したゴールの駅に近づいても通知は発火しない
```

UC-05: ルートパターンを管理する

目的:　よく使う複数目的地のルートをパターンとして保存し、ワンタップで適用する
利用者:　エンドユーザー
事前条件:　アプリが起動済みである

基本フロー（パターン登録）:
1. マルチゴールを2件以上登録した状態でマルチゴール一覧ダイアログを開く
2. 「パターン登録」ボタンが表示される（2件未満のときは非表示）
3. ユーザーが「パターン登録」ボタンをタップする
4. システムが現在のマルチゴール駅リストをSharedPreferencesに保存する（キーはタイムスタンプ）
5. 「登録しました」ダイアログで駅順（A → B → C形式）を確認できる

基本フロー（パターン選択・適用）:
1. ユーザーがマルチゴール一覧ダイアログの「パターン」ボタンをタップする
2. システムがパターン一覧ダイアログを表示する
3. ユーザーが一覧からパターンのCircleAvatarをタップする（赤くハイライト＝装填状態）
4. ユーザーが「設定」ボタンをタップする
5. システムが既存の全ジオフェンス・マルチゴールエントリをクリアする
6. システムが選択パターンの各駅を新たなマルチゴールとして登録する（駅名・座標を保存）
7. 各駅のジオフェンスを登録する
8. ダイアログが閉じ、マルチゴール一覧に選択パターンの駅が表示される

削除フロー:
1. パターン一覧ダイアログにて削除ボタン（ゴミ箱）をタップする
2. システムがSharedPreferencesからそのパターンを削除する
3. 削除したパターンが選択中だった場合はハイライトをクリアする
4. 一覧が更新される

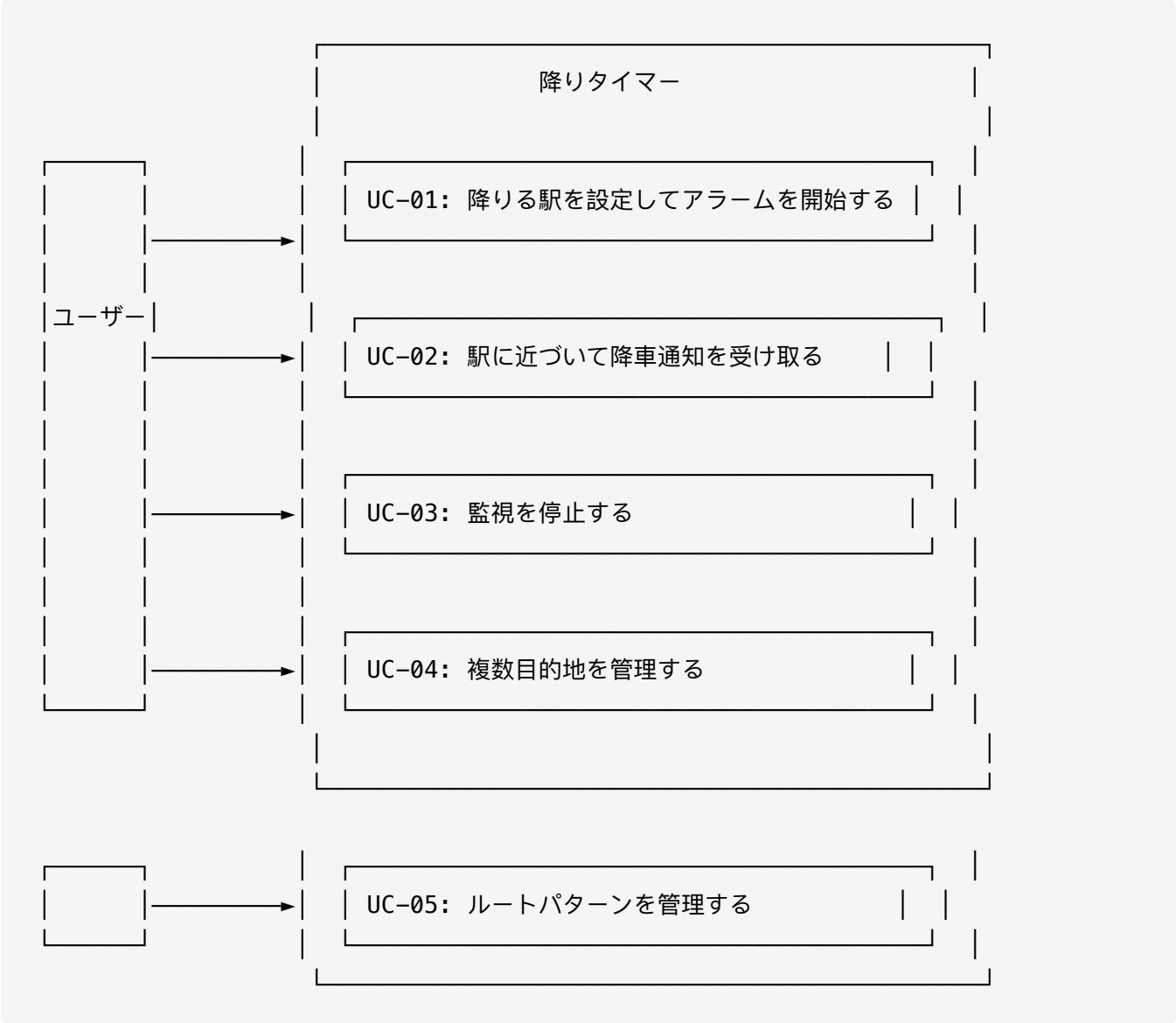
代替フロー:
4a. パターンが選択されていない場合 → 「設定」ボタンを押しても何もしない

事後条件: 選択したパターンの全駅のジオフェンスが登録され、監視が開始される

UC-04: 複数目的地を管理する

```
目的:        複数の降車駅を事前に登録・確認・削除する
利用者:      エンドユーザー
事前条件:    アプリが起動済みである

基本フロー:
  1. ユーザーがホーム画面の「複数」ボタンをタップする
  2. システムがマルチゴール一覧ダイアログを表示する
  3. ユーザーが「設定」ボタンをタップする
  4. システムがマルチゴール設定ダイアログを表示する
  5. ユーザーが空きスロット（番号）と駅を選択する
  6. ユーザーが「設定」ボタンをタップして登録する
  7. システムが駅名・座標をSharedPreferencesに保存し、ジオフェンスを登録する
  8. 登録済み一覧に追加されて表示される

代替フロー:
  3a. 既に10件登録済みの場合
      → 「目的地は10個までしか登録できません」エラーを表示する
  6a. 番号または駅が未選択の場合
      → 「番号と駅を選択してください」エラーを表示する

削除フロー:
  1. マルチゴール一覧ダイアログにて、最後に登録したゴールの削除ボタン（ゴミ箱）をタップする
  2. システムがジオフェンスを削除し、SharedPreferencesからエントリを削除する
  3. 一覧が更新される

事後条件: 登録したゴールに近づくとジオフェンスイベントが発火し、降車通知が届く
```

ユースケース図



---

6. 機能要件

> **このセクションの目的**：アプリに「何ができてほしいか」を要望レベルで列挙する。基本設計書の機能一覧（F-01～）の元になるリスト。
>
> **補足**：要件定義書では「通知が欲しい」のような要望レベルで書く。それを「NativeGeofenceManagerでentryイベントを登録する」のように具体化するのは基本設計書・詳細設計書の役割。

6.1 要件一覧

要件ID	カテゴリ	要件
REQ-01	表示	東京の主要路線と駅の一覧を表示できる
REQ-02	操作	一覧から降りたい駅を選択できる
REQ-03	監視	選択した駅のジオフェンス監視を開始できる
REQ-04	通知	選択した駅の近くに到達したらプッシュ通知を受け取れる
REQ-05	操作	ジオフェンス監視を任意のタイミングで停止できる
REQ-06	権限	位置情報パーミッション︵常に許可︶を取得できる
REQ-07	権限	通知パーミッションを取得できる
REQ-08	表示	現在どの駅を監視対象に選択しているかが分かる
REQ-09	表示	監視が起動中かどうかの状態が画面で分かる
REQ-10	通知	到着通知にバイブレーションを伴い｡ユーザーが停止するまでループし続ける
REQ-11	状態管理	アプリを再起動しても監視状態︵目のアイコンのON/OFF︶と選択駅名の両方
REQ-12	通知	ジオフェンス侵入時に音楽ストリームの音量を最大に上げる︵Android のみ︶｡音量はユーザーが手動で調整する
REQ-13	操作	駅名を入力して前方一致で検索し｡該当する路線へ画面をスクロールしてジャ
REQ-14	操作	選択駅が選ばれている場合｡ボタンを押すことでその駅を含む最初の路線へリ
REQ-15	操作	同名駅が複数路線に存在する場合｡ボタンを押すたびに次の路線の同名駅へ順
REQ-16	操作	最大10件の目的地を個別に登録･表示･削除でき｡それぞれのジオフェンスが同
REQ-17	状態管理	アプリを再起動してもマルチゴールのジオフェンスが自動的に復元される

要件ID	カテゴリ	要件
REQ-18	操作	2件以上のマルチゴールをルートパターンとして保存できる
REQ-19	操作	保存済みのパターンを選択し、マルチゴールとして一括適用できる。ジオフェ
REQ-20	表示	マルチゴールが1件になった時、単一選択と同じ駅名表示・ジャンプボタン操作

6.2 優先度の定義

優先度	定義	判断基準
必須 (Must)	この要件が満たされないとリリースできない	アプリの核となる機能
重要 (Should)	可能な限り実装すべき	ユーザー体験を大きく向上させる
任意 (Could)	あれば嬉しい	なくてもアプリとして成立する

> **補足：** この優先度分類は「MoSCoW法」と呼ばれる手法に基づいている。Must / Should / Could / Won't の頭文字を取ったもの。企業の開発現場でよく使われる。

---

7. 非機能要件

> **このセクションの目的：** 「機能」以外の品質に関する要件を定義する。
>
> **補足：** 「通知が届く」は機能要件。「駅に入ってから30秒以内に通知が届く」は非機能要件。どちらも重要だが、非機能要件は見落とされがちなので、意識的に書き出す必要がある。

7.1 性能

要件ID	要件	基準値	備考
NFR-01	アプリ起動から駅一覧表示まで	3秒以内	ネットワーク環境に依存
NFR-02	ジオフェンス登録処理	2秒以内	ネイティブAPIの応答に依存
NFR-03	駅圏内進入から通知発火まで	可能な限り短時間	OS·省電力設定に依存するため厳密

7.2 可用性︵アベイラビリティ︶

> **補足** ：可用性とは︵使いたいときに使えること︶。特にバックグラウンド動作は、OSによる制限を受けやすいため、要件として明記しておくことが重要。

要件ID	要件
NFR-04	アプリがバックグラウンド状態·終了状態でもジオフェンス監視が継続すること
NFR-05	APIサーバーが停止していてもアプリはクラッシュしないこと
NFR-06	通信エラー時はエラーメッセージを表示し、アプリが利用継続できること

7.3 ユーザビリティ︵使いやすさ︶

要件ID	要件
NFR-07	駅選択から監視開始まで3タップ以内で操作できること
NFR-08	現在の選択駅と監視状態が画面上で一目で分かること
NFR-09	到着通知のバイブレーションはユーザーが停止するまでループし続けること
NFR-10	アプリを再起動した後も監視中状態︵目のアイコンの色︶と選択駅名が維持されること
NFR-10	パーミッションが未取得の場合、適切なガイダンスを表示すること

7.4 保守性︵メンテナンスのしやすさ︶

> **補足** ：保守性とは︵後から修正·拡張がしやすいこと︶。今は個人開発でも、半年後の自

> 分が﹁このコードどういう意味だっけ﹂とならないために重要。

要件ID	要件	理由
NFR-11	API接続先は1箇所で変更可能にすること	サーバー移行時の影響を最小化す
NFR-12	画面とデータ取得ロジックは分離すること	変更時の影響範囲を限定するため
NFR-13	データモデルはコード生成で型安全性を担保すること	手書きによるバグを防ぐため
NFR-14	ファイルは役割ごとにディレクトリを分割すること	目的のファイルを見つけやすくす

7.5 セキュリティ

要件ID	要件	現状
NFR-15	位置情報は通知の発火にのみ使用し、外部送信しないこと	設計通り（ローカル処理のみ
NFR-16	APIとの通信	HTTP（非暗号化）
NFR-17	アクセス制御	なし（認証不要）

7.6 互換性

要件ID	要件
NFR-18	Android と iOS の両方で動作すること
NFR-19	各プラットフォームのジオフェンスAPIの制約に対応すること

---

8. 制約条件

> **このセクションの目的**：開発における﹁前提﹂や﹁制限﹂を明記する。制約を把握しておくことで、実現不可能な要件を初期段階で排除できる。
>
> **補足**：制約条件には﹁技術的な制約﹂﹁コスト・リソースの制約﹂﹁外部の制約﹂があ

> る。

8.1 技術的制約

#	制約	理由
1	フロントエンドは Flutter で開発する	学習目的。iOS·Android のクロスプラットフォーム対応
2	状態管理は Riverpod を使用する	Flutter公式推奨に準拠した学習
3	データモデルは freezed を使用する	型安全性とイミュータブル性の担保
4	ジオフェンスは native_geofence パッケージを使用する	Flutterからネイティブジオフェンスを利用する
5	位置情報の「常に許可」パーミッションが必須	バックグラウンドでのジオフェンス監視に必要

8.2 OSレベルの制約

> **補足**：ジオフェンスはOSのネイティブ機能を利用する。特にiOSとAndroidではその制約が異なるため、要件定義段階で把握しておく必要がある。

#	制約	内容
1	iOS: バックグラウンド処理制限	iOSのバックグラウンドモードの制限を受ける
2	Android: 省電力モード	Dozeモードや省電力設定によりジオフェンスイベントが遅延する
3	両OS: 通知パーミッション	ユーザーが通知を許可しなければ通知が届かない
4	両OS: 位置情報パーミッション	「常に許可」がなければバックグラウンドでのジオフェンス監視が

8.3 リソース制約

#	制約	内容
1	開発者	1名（個人開発）

#	制約	内容
2	予算	無料の範囲︵サーバーは既存のものを利用︶
3	開発ツール	AI︵Claude︶をペアプログラミング的に活用

8.4 外部制約

#	制約	内容
1	駅データ	外部APIサーバー (toyohide.work) から取得する
2	ネットワーク	初回起動時のデータ取得にはインターネット接続が必要
3	対応路線	APIが提供する東京の路線·駅データに限定される

---

9. リスクと対策

> **このセクションの目的**：﹁プロジェクトの途中で何が問題になりそうか﹂を事前に洗い出し、対策を考えておく。
>
> **補足**：リスクとは﹁将来起こるかもしれない問題﹂のこと。起きてから対処する︵事後対応︶よりも、事前に対策を打っておく︵予防︶方がはるかにコストが低い。

リスクID	リスク	影響度	発生確率
R-01	ジオフェンスイベントがOSの省電力設定で遅延·無効化される	高	中
R-02	ユーザーが﹁常に許可﹂パーミッションを拒否する	高	中
R-03	駅データAPIサーバーが停止する	中	低
R-04	iOSとAndroidでジオフェンスの挙動が異なり品質にばらつきが出る	中	高
R-05	バッテリー消費が増加してユーザー体験を損なう	中	中
R-06	半年後にコードの意味を忘れる	中	高

> **補足**：影響度×発生確率で優先度を判断する。R-04（プラットフォーム差異）のように、影響度・発生確率が共に高いものは最優先で対策を打つべき。

---

10. 受入条件

> **このセクションの目的**：「何をもって完成とするか」を定義する。受入条件がないと「まだ終わってないのでは」「もう少し改善できるのでは」と、いつまでも完了にならない。
>
> **補足**：企業のプロジェクトでは、受入条件を満たしたことを発注者が確認して「検収」となる。個人開発でも「ここまでできたら一区切り」を決めておくと、達成感を得やすい。

10.1 機能面の受入条件

#	条件	検
1	路線・駅の一覧が表示される	アプリ起動後に路線ごとにグループ化された駅一覧が表示され
2	駅を選択できる	駅名をタップすると選択状態（ハイライト）になることを確認
3	ジオフェンス監視を開始できる	監視開始ボタンをタップし、監視中の状態表示に変わることを
4	駅圏内で通知が発火する	テスト用の座標を設定し、ジオフェンス内進入時に通知が届く
5	監視を停止できる	停止ボタンをタップし、ジオフェンスが解除されることを確認
6	パーミッション要求が動作する	要求ボタンをタップし、OS標準のパーミッションダイアログが
7	駅名検索が動作する	検索フォームに駅名を入力して検索ボタンをタップし、ダイア

10.2 非機能面の受入条件

#	条件	検証
1	アプリ起動から3秒以内にデータ表示	Wi-Fi環境でストップウォッチ計測
2	バックグラウンド状態でジオフェンスが機能する	アプリをバックグラウンドにした状態でジオ

#	条件	検証
3	APIエラー時にクラッシュしない	サーバーが停止した状態でアプリを起動し｀
4	Android実機で正常動作する	Android実機でテスト

10.3 現在のステータス

> **補足**：受入条件の達成状況を記録しておくと｀プロジェクトの進捗が一目でわかる｡

#	条件	ステータス
機能1~6	全機能の受入条件	**達成済み**
非機能1~4	全非機能の受入条件	**達成済み**

付録A: 用語集

> **補足**：要件定義書は非技術者も読む可能性があるため｀技術用語やドメイン用語を定義しておく｡

用語	意味
ジオフェンス	地図上に仮想的な境界線︵フェンス︶を設定し｀デバイスがその範囲に入っ
乗り越し	電車で目的の駅を通り過ぎてしまうこと
ステークホルダー	プロジェクトに関係する全ての人｡発注者｀開発者｀ユーザーなど
スコープ	プロジェクトの対象範囲｡﹁作るもの﹂と﹁作らないもの﹂の境界線
スコープクリープ	プロジェクトの進行中に範囲が際限なく膨らむこと｡予防が重要
ユースケース	ユーザーがシステムを使って目的を達成する一連の流れ
MoSCoW法	優先度分類手法｡Must︵必須︶/ Should︵重要︶/ Could︵任意︶/ Won't︵対
非機能要件	機能以外の品質要件｡性能｀可用性｀セキュリティなど

用語	意味
受入条件	﹁完成﹂と判断するための基準。これを満たしたらプロジェクト完了
リスク	将来起こるかもしれない問題。事前に対策を打つことが重要
パーミッション	アプリがOSの機能︵位置情報·通知など︶を利用するためのユーザー許可
バックグラウンド動作	アプリが画面に表示されていない状態︵または終了状態︶でも機能すること
Riverpod	Flutter向けの状態管理ライブラリ
freezed	Dartのコード生成ライブラリ。イミュータブル︵変更不可︶なモデルクラス
REST API	HTTPプロトコルでデータをやりとりする設計パターン

付録B: ドキュメント体系

> この付録は変更しないでください。付録Cに仕様書変更ガイドがあります。

> **補足**：このプロジェクトのドキュメントがどう関連しているかを図示する。

```
requirements.md（この文書）
要件定義書
「なぜ作るか」「何が必要か」
      │
      ▼
basic-design.md
基本設計書
「何を作るか」「どう振る舞うか」
      │
      ▼
detailed-design.md
詳細設計書
「どう作るか」「具体的な実装仕様」
      │
      ▼
ソースコード
lib/ 以下の Dart ファイル
```

ドキュメント	ファイル	主な読者
要件定義書	`documents/requirements.md`	発注者·PM·開発者
基本設計書	`documents/basic-design.md`	PM·開発リーダー·開発者
詳細設計書	`documents/detailed-design.md`	開発者

付録C: 仕様書変更ガイド

> **このセクションの目的**：「コードを変更したとき、どの仕様書のどこを更新すればよいか」を明確にする。
> 仕様書はコードと常に一致していなければ、読んでも信用できないドキュメントになってしまう。

C.1 変更の種別と更新すべき仕様書

変更の内容によって、更新が必要な仕様書の範囲が変わる。

変更の種別	具体例
ユーザー向けの振る舞いが変わる	通知が1回→ループに変わる、バイブレーションの感知距離
新機能を追加する	SharedPreferencesで状態を永続化する
数値パラメータが変わる（内部）	ジオフェンス半径: 500m→1000m
使用パッケージが変わる	`vibration` パッケージを追加する
内部実装のみ変わる（振る舞いは同じ）	関数名のリファクタリング
バグ修正（仕様変更なし）	ボタンが押せないバグを直す

> **判断の原則：**
>
> - 「ユーザーから見た振る舞いが変わるか？」→ 変わるなら要件定義書と基本設計書も更新する
> - 「何を作るかが変わるか？」→ 変わるなら基本設計書も更新する
> - 「どう作るかが変わるか？」→ 変わるなら詳細設計書を更新する

C.2 各仕様書で更新するセクション一覧

要件定義書（`requirements.md`）を更新するとき

変更した内容	更新するセクション
機能の振る舞いが変わった	5. 業務フロー（該当ユースケース）、6.1 要件一覧
品質に関わる条件が変わった	7. 非機能要件（該当項目）
ジオフェンスの感知距離が変わった	2.2 達成指標、5. UC-02

変更した内容	更新するセクション
新機能が増えた	6.1 要件一覧︵新規REQ追加︶ ､7. 非機能要件︵必要なら︶
すべての変更共通	改訂履歴︵版･日付･内容を追記︶

基本設計書（`basic-design.md`）を更新するとき

変更した内容	更新するセクション
機能の振る舞いが変わった	3. 機能一覧 ､4. 機能要件︵該当機能のF-xx︶
新機能が増えた	3. 機能一覧︵新規F-xx追加︶ ､4. 機能要件︵詳細記述︶
データの保持方法が変わった	6.3 データの取得タイミング
画面の表示･操作が変わった	5. 画面一覧と画面遷移
技術スタック︵パッケージ︶が変わった	8. 非機能要件 / システム構成
ジオフェンスの距離が変わった	1.2 目的 ､2.4 ジオフェンスの仕組み ､4. F-04
すべての変更共通	改訂履歴

詳細設計書（`detailed-design.md`）を更新するとき

変更した内容	更新するセクション
パッケージが増えた	2.1 技術スタック
データモデルが変わった	4. データモデル定義︵該当クラス︶
Provider･Notifierのメソッドが変わった	5. 状態管理設計
画面の初期化処理が変わった	6.2 ホーム画面︵初期化処理︶
ジオフェンスコールバックが変わった	6.3 ジオフェンスコールバック
ジオフェンスの設定値が変わった	8. ジオフェンス仕様︵該当項目︶
すべての変更共通	改訂履歴

C.3 今回の変更を例に見るガイドの使い方

今回︵v1.1︶は以下の4つの変更を行った。それぞれどの仕様書のどこを更新したかを示す。

変更① バイブレーションをループ化︵ユーザーが止めるまで繰り返す︶

- **変更の種別**：ユーザー向けの振る舞いが変わる → 3つの仕様書すべて更新

仕様書	更新箇所	
要件定義書	REQ-10の内容更新、UC-02の基本フロー更新、NFR-09更新	ユーザー体験︵
基本設計書	F-05の振る舞い更新、F-06の停止時の振る舞い追記	﹁何ができるか
詳細設計書	geofenceCallback関数の更新、バイブレーションパターン表の更新	実装方法が変わ

変更② ジオフェンス半径 500m → 1000m

- **変更の種別**：数値パラメータが変わる → 3つの仕様書すべて更新︵目的が変わるほど大きな数値変更のため︶

仕様書	更新箇所	理由
要件定義書	2.2 達成指標、UC-02の検知距離	受入条件の数値が変わるため
基本設計書	1.2 目的、2.4 ジオフェンスの仕組み、F-04	機能の仕様値が変わるため
詳細設計書	8.1 共通設定︵radiusMeters︶、テスト方法	実装コードの設定値が変わるため

変更③ バイブレーション強度を最大化︵intensities: 255︶

- **変更の種別**：内部実装のみ変わる︵振る舞いの﹁強さ﹂は変わるが機能の本質は変わらない︶

仕様書	更新箇所	理
要件定義書	❌ 不要	﹁バイブレーションで知らせる﹂と

仕様書	更新箇所	理
基本設計書	❌ 不要	﹁強くする﹂は実装レベルの詳細で
詳細設計書	6.3 バイブレーションパターン表︿intensities追加︾	コードレベルの仕様が変わるため

変更④ SharedPreferencesによる監視状態の永続化

- **変更の種別**：新機能の追加 → 3つの仕様書すべて更新

仕様書	更新箇所
要件定義書	REQ-11を新規追加｡NFR-10を新規追加
基本設計書	機能一覧にF-10追加｡6.3 データの取得タイミングに SharedPreferences 追記
詳細設計書	AppParamStateにメソッド追記｡appParamProviderの詳細更新｡_initPluginsの初期化処

改訂履歴

版	日付	
1.0	2026-02-28	初版作成
1.1	2026-03-01	ジオフェンス半径を500m→1000mに変更︿2.2·UC-02︾｡バイブレーションのパ
1.2	2026-03-03	選択駅の永続化をREQ-11·NFR-10に追記｡停止時に選択駅も消去される仕様をU
1.3	2026-03-03	ジオフェンス侵入時の音量最大化機能を追加︿REQ-12新規追加·UC-02基本フロ
1.4	2026-03-07	駅名検索機能を追加︿REQ-13新規追加·UC-01基本フロー更新·代替フロー3a追加
1.5	2026-03-08	選択駅ジャンプ機能を追加︿REQ-14新規追加︾｡複数路線同名駅の循環ジャン
1.6	2026-03-14	複数目的地登録機能を追加︿REQ-16·REQ-17新規追加︾｡UC-03を停止ボタン
1.7	2026-03-15	ルートパターン管理機能を追加︿REQ-18·REQ-19·REQ-20新規追加︾｡UC-05